1. Problem Statement

1.1 Context & Domain Background

In the rapidly evolving digital education landscape, modern e-learning ecosystems require platforms capable of combining seamless video delivery, transactional security, and verifiable academic credentialing. Traditional learning management systems (LMS) often rely on legacy monolithic architectures where the user interface, business logic, and database access are tightly coupled. This structural dependency creates significant operational bottlenecks when scaling services to accommodate large numbers of concurrent users.

1.2 Key Identified Technical Pain Points

Architectural Monolithism & High Maintenance Overhead

Legacy monolithic codebases suffer from severe coupling. A single modification in the data model can lead to cascading failures across UI components and business services.

Testing and deploying isolated features requires redeploying the entire application, resulting in long downtime windows and high operational risk.

Media Bandwidth Saturation & Content Security Risks

Storing heavy video assets directly on web application servers causes disk space exhaustion and bandwidth saturation during peak usage.

Serving static video links without tokenized protection leaves intellectual property vulnerable to unauthorized hotlinking, scrapers, and distribution outside paid enrollments.

Transactional Inconsistencies During Checkout & Enrollment

High-traffic checkout workflows using asynchronous payment gateways (e.g., Stripe) frequently face race conditions and state mismatches.

Without transactional unit-of-work encapsulation, failures during post-payment callbacks can result in orphaned financial records—where a student's card is charged, but enrollment records fail to persist in the database.

Lack of Cryptographic Authenticity in Course Certification

Standard e-learning platforms often issue unverified static PDF certificates or HTML render templates that can easily be manipulated or forged using browser inspection tools.

Employers and academic institutions lack an automated, tamper-proof mechanism to independently verify the authenticity of a student's certificate.

1.3 Proposed Solution Vision

EduHub addresses these architectural limitations by implementing an enterprise-grade N-Tier architecture built on ASP.NET Core MVC. The solution isolates operational responsibilities across dedicated layers, offloads video ingestion and streaming to Microsoft Azure Blob Storage using short-lived Shared Access Signature (SAS) tokens, guarantees ACID transaction integrity through the Unit of Work pattern, and provides automated, cryptographically verifiable PDF certificates using iTextSharp and unique validation hashes.

2. Project Objectives

2.1 Core Engineering & Business Goals

The primary objective of EduHub is to engineer a resilient, maintainable, and cloud-native e-learning platform that satisfies enterprise software engineering benchmarks. The key engineering goals are structured as follows:

Architectural Decoupling & Clean Layering

Establish a strict N-Tier Architecture separating Presentation (`EduHub.Web`), Business Logic (`EduHub.Services`), Data Access (`EduHub.Data`), and Core Domain Models (`EduHub.Core`).

Enforce SOLID design principles to ensure high maintainability, loose coupling, and comprehensive testability using Dependency Injection.

Transactional Integrity & Data Consistency

Implement the Generic Repository and Unit of Work patterns over Entity Framework Core to guarantee atomic execution across complex database operations.

Eliminate orphaned records across multi-item checkouts, coupon redemptions, and student course enrollments.

Cloud-Native Media Ingestion & Tokenized Streaming

Integrate Microsoft Azure Blob Storage for secure, direct-to-cloud media hosting.

Implement time-limited Shared Access Signature (SAS) tokens to restrict video streaming access strictly to authorized, active learners.

Extensible Design Pattern Execution

Utilize the **Strategy Pattern** to support pluggable pricing engines, seasonal promotions, and dynamic voucher calculations without modifying core checkout logic.

Leverage the **Observer Pattern** to capture course completion events asynchronously, automatically updating student progress records and triggering certificate issuance.

Automated Verifiable Credentialing

Automate server-side dynamic compilation of course completion certificates in PDF format using iTextSharp.

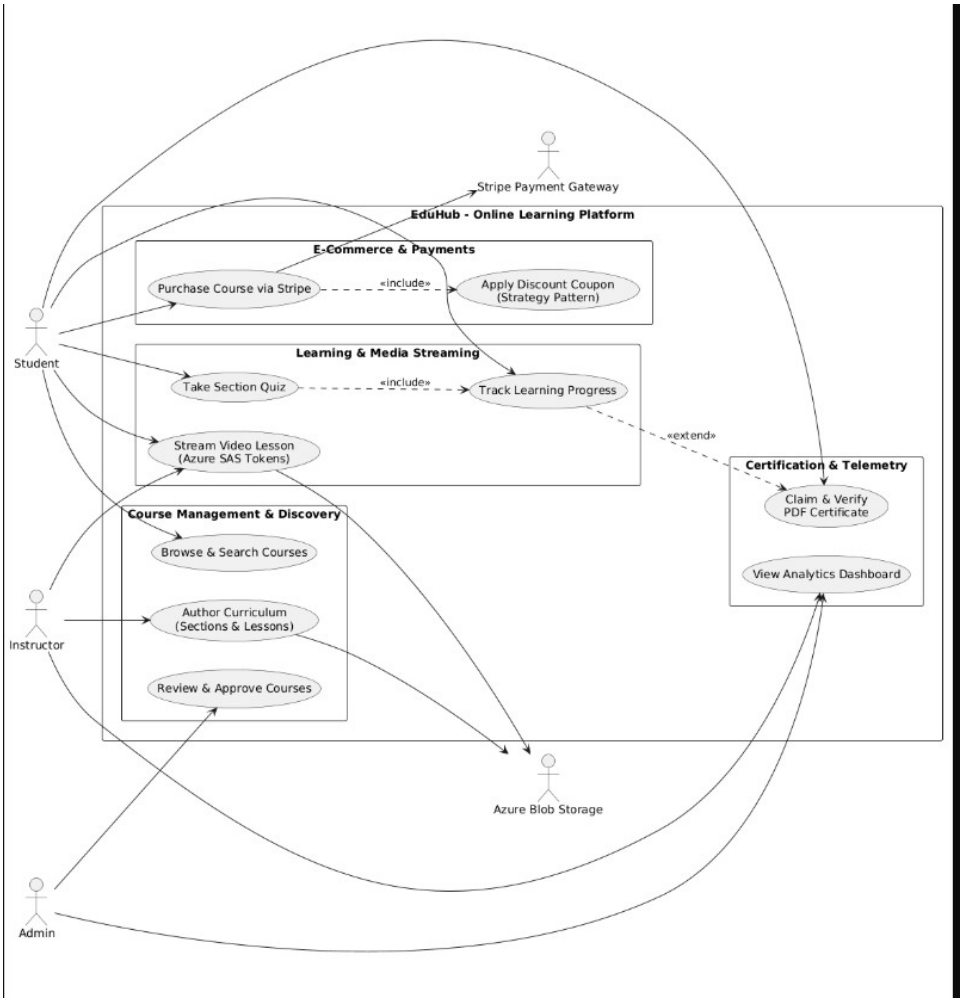
Embed cryptographic hashes (SHA-256) and public verification QR codes into each generated document for instant verification by third parties.

Intelligent AI Learning Assistance & Telemetry

Incorporate AI-driven learning tools including automated lecture summarization, dynamic quiz generation, and semantic course catalog searching.

Provide administrators and instructors with real-time interactive telemetry dashboards powered by Chart.js.

3- Use-Case-Diagram



4. Use Case Descriptions

UC-01: Browse & Search Course Catalog

Primary Actor: Student / Guest Visitor

Goal: Search and filter the course catalog by keywords, categories, rating, and price to locate relevant material.

Preconditions: User is navigating the platform's public portal or catalog page.

Main Success Scenario:

User enters keywords in the search bar or selects specific category filters.

System executes an optimized, indexed database query.

System renders matching course cards displaying title, instructor name, rating, duration, and price with sub-second response times.

Alternative Flows:

1a. No matches found: System displays a notification indicating no results were found and suggests alternative terms or related categories.

UC-02: Purchase Course via Stripe Checkout

Primary Actor: Authenticated Student

Goal: Securely purchase one or multiple courses in a single checkout session.

Preconditions: Student is logged in and has at least one active item in the shopping cart.

Main Success Scenario:

Student initiates checkout from the shopping cart.

System evaluates cart totals and applies active discount coupons using the **Strategy Pattern**.

System creates a secure Stripe Checkout session and redirects the client browser.

Student submits payment details securely on the Stripe payment page.

Stripe sends an asynchronous webhook event (`checkout.session.completed`) back to EduHub.

System opens a **Unit of Work** transaction to persist `Order`, `OrderDetails`, and `Enrollment` records atomically.

Transaction commits, and the student receives instant access to the course content.

Alternative Flows:

5a. Payment declines: Stripe redirects student back to cart with an error message; no database mutations occur.

UC-04: Stream Video Lesson & Log Progress

Primary Actor: Enrolled Student

Goal: Stream lesson videos securely while the platform tracks viewing progress automatically.

Preconditions: Student is authenticated and actively enrolled in the course.

Main Success Scenario:

1. Student selects a lesson from the course curriculum navigation tree.
2. System verifies active enrollment and requests a short-lived Shared Access Signature (SAS) token from Azure Blob Storage.
3. Video player (Video.js) initializes playback using the secure SAS URL.
4. Upon reaching complete playback or manual confirmation, client events trigger progress logging API endpoints.
5. System updates progress state in the database and recalculates overall course percentage.

Postconditions: Lesson status updates to completed, and progress bar reflects new percentage.

UC-07: Author Course Curriculum

Primary Actor: Instructor

Goal: Create and structure course sections, lessons, and media resources.

Preconditions: Instructor is logged in with valid content authoring permissions.

Main Success Scenario:

1. Instructor navigates to the Instructor Studio and opens the Course Builder interface.
2. Instructor creates sections, defines lesson titles, and attaches video files or supplemental documents.
3. System streams video files directly to Azure Blob Storage containers.
4. Instructor reviews curriculum structure and submits the course for administrative review.

Postconditions: Course state transitions to `PendingApproval` awaiting administrator audit.

UC-09: Claim & Verify Dynamic PDF Certificate

Primary Actor: Enrolled Student / Public Verifier

Goal: Issue a verifiable PDF certificate upon course completion and allow third parties to validate its authenticity.

Preconditions: Student achieves 100% completion across all lessons and achieves passing grades on required quizzes.

Main Success Scenario:

Observer Pattern event listener detects 100% completion trigger.

System compiles a dynamic PDF certificate using **iTextSharp**, embedding student name, course title, completion timestamp, unique certificate GUID, and QR code.

PDF asset is saved to cloud storage, and a download link is attached to the student profile.

An employer scans the certificate QR code or inputs the GUID into the public verification portal.

System looks up the unique GUID and displays official confirmation of certificate validity.

5. Software Architecture

5.1 Architectural Strategy

EduHub is built upon a strict **N-Tier (Layered) Architecture** integrated with **Domain-Driven Design (DDD)** concepts. The application isolates responsibilities across four logical layers, preventing UI controllers from executing raw database queries or embedding complex business logic.

5.2 Architectural Layer Breakdown

1. Presentation Layer (`EduHub.Web`)

-

Responsibilities: Renders the user interface, handles incoming HTTP requests, performs input model validation, and returns Razor views or API JSON payloads.

Technologies: ASP.NET Core MVC, Bootstrap 5, Video.js for media playback, and Chart.js for analytical dashboards.

2. Business Logic Layer (`EduHub.Services`)

Responsibilities: Encapsulates business rules, orchestrates application workflows, executes design pattern strategies, and handles external cloud integrations.

Applied Patterns:

Strategy Pattern: Swaps pricing algorithms (percentage discounts, fixed amounts) dynamically during checkout.

Observer Pattern: Listens for lesson progress updates and fires downstream certificate compilation events upon reaching 100% progress.

3. Data Access Layer (`EduHub.Data`)

Responsibilities: Manages entity persistence, database migrations, and transactional boundaries over Azure SQL Database.

Applied Patterns:

Repository Pattern: Abstracts EF Core data access logic behind domain interfaces.

Unit of Work Pattern: Wraps repository operations inside atomic transactions to ensure data consistency across complex business operations.

4. Core Domain Layer (`EduHub.Core`)

Responsibilities: Defines core domain models, interfaces, enums, DTOs, and system specifications. Contains zero dependencies on outer layers or framework assemblies.

